SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I **N.SANDIYANITTIANANDAM(192571029)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
SANDIYA

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that: Constraints:

   i. D1 cannot work Night shift ii. D2 must work before D3 (Morning < Afternoon < Night) iii. Only one doctor per shift iv. D3 cannot work Morning

   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved. c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Hospital Doctor Shift Assignment Using Backtracking Search

Problem Statement

A hospital needs to assign three doctors (D1, D2, D3) to three shifts: Morning, Afternoon and Night. The assignment must satisfy all the given constraints.

Constraints

1. D1 cannot work the Night shift.
2. D2 must work before D3.
3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. Every doctor must be assigned exactly one shift.

Solution Using Backtracking Search

Step 1: Initialize the variables

Let D1, D2 and D3 represent the doctors. Each doctor can be assigned one of the three shifts.

Step 2: Assign D2 to the Morning shift Assign D2 = Morning.

Constraint check: This assignment is valid because D2 can work the Morning shift.

Step 3: Assign D3 to the Night shift Assign D3 = Night.

Constraint check: D3 is not assigned to the Morning shift, and D2 works before D3. Therefore, both constraints are satisfied. **Step 4: Assign D1 to the Afternoon shift** Assign D1 = Afternoon.

Constraint check: D1 is not assigned to the Night shift, and each shift has only one doctor. Therefore, this assignment is valid.

Step 5: Verify all constraints

Constraint	Verification	Status
D1 cannot work Night	D1 works Afternoon	Satisfied
D2 must work before D3	Morning comes before Night	Satisfied
One doctor per shift	Each shift has one doctor	Satisfied
D3 cannot work Morning	D3 works Night	Satisfied
Every doctor gets one shift	All doctors are assigned	Satisfied

Final Valid Schedule

Shift	Assigned Doctor
Morning	D2
Afternoon	D1
Night	D3

Result: Backtracking Search finds a valid schedule by assigning doctors to shifts and checking each constraint. The final assignment satisfies all the given conditions.

2. Robot Navigation in a 5×5 Grid Using BFS

Problem Statement

A robot must move from the Start (S) to the Goal (G) in a 5×5 grid. It can move Up, Down, Left and Right. Each move costs 1, and the robot cannot pass through obstacles.

Assumption: Since the grid diagram is not visible in the extracted question paper, the following example grid is used for demonstration.

Example Grid

S = Start, G = Goal, X = Obstacle, and . = Free cell.

S X . . .
. X . . .
. X . . .
. X . . .
. . . . G

Manhattan Distance Heuristic

The Manhattan distance estimates the distance between the current cell and the goal.

$$h(n) = | x_n - x_G | + | y_n - y_G |$$

Here, x_n, $y_n$are the coordinates of the current cell, and x_G, $y_G$are the coordinates of the goal.

For the Start at (1,1) and Goal at (5,5):

$$h(S) = | 1 - 5 | + | 1 - 5 | = 8$$

Breadth-First Search (BFS) Algorithm

BFS explores nodes level by level using a FIFO queue. It finds the shortest path when all movements have equal cost. **Steps:**

1. Insert the Start node into the queue.
2. Remove the front node from the queue.
3. Check whether it is the Goal.
4. Generate its valid adjacent cells in the order Up, Down, Left and Right.
5. Reject cells that are obstacles or already visited.
6. Add the remaining cells to the queue and record their parent nodes.
7. Repeat until the Goal is reached.
8. Trace the parent nodes backwards to obtain the optimal path.

Step-by-Step Node Expansion

Coordinates are represented as (row, column).

Step	Expanded Node	Valid New Nodes Added	Queue After Expansion

1	(1,1)	(2,1)	(2,1)
2	(2,1)	(3,1)	(3,1)
3	(3,1)	(4,1)	(4,1)
4	(4,1)	(5,1)	(5,1)
5	(5,1)	(5,2)	(5,2)
6	(5,2)	(5,3)	(5,3)
7	(5,3)	(5,4)	(5,4)
8	(5,4)	(5,5)	(5,5)
9	(5,5)	Goal reached	Empty

Priority Queue and Heuristic Note

Standard BFS uses a FIFO queue, not a priority queue. Manhattan distance can be calculated for each node, but it does not change the BFS expansion order. If nodes are instead selected using a priority queue based on $f(n) = g(n) + h(n)$, the algorithm becomes A* Search. Therefore, BFS is used here to determine the shortest path, while Manhattan distance is shown as the requested heuristic.

Optimal Path

The robot follows this path:

$$(1, 1) \rightarrow (2, 1) \rightarrow (3, 1) \rightarrow (4, 1) \rightarrow (5, 1) \rightarrow (5, 2) \rightarrow (5, 3) \rightarrow (5, 4) \rightarrow (5, 5)$$

Total Cost:

$$\text{Cost} = 8 \times 1 = 8$$

Result: BFS finds an optimal path with a total cost of 8 for the example grid. The exact path and expansion order may differ if the original question paper uses a different grid.

3. Autonomous Rescue Robot in a Disaster-Affected Building

a) Analysis of the Constraints

The rescue robot must reach a survivor while avoiding obstacles and minimizing the total movement cost.

1. **Movement constraints:** The robot can move only Up, Down, Left and Right. It cannot move diagonally.
2. **Obstacles:** Blocked cells cannot be entered. The robot must find an alternative route when a path is blocked.
3. **Limited sensing:** The robot can observe only adjacent cells. It does not have complete information about the building and must explore unknown areas.
4. **Movement cost:** Each normal movement costs 1, while entering a risky zone adds an additional cost of 2. Therefore, entering a risky zone has a total cost of 3.
5. **Cost minimization:** The robot must select a safe route with the lowest total cost rather than simply choosing the route with the fewest movements.
6. **Dynamic knowledge:** The robot must update its internal map whenever it discovers a new obstacle, a risky zone or a previously unknown passage.

These constraints require the robot to make decisions using incomplete information and revise its plan whenever new information becomes available. **b) Solution**

Using Uniform Cost Search and Online Search

Uniform Cost Search (UCS) is suitable for finding the minimum-cost path when the relevant map and movement costs are known. An online search agent is needed to explore when the map is initially unknown.

The robot can combine online exploration with UCS on the currently known map.

Step 1: Initialize the robot

The robot starts at position S and sets its initial movement cost to zero.

Step 2: Observe the surroundings

The robot senses the four adjacent cells and identifies whether they are free, blocked or risky.

Step 3: Update the internal map

The robot records the newly discovered cells and their movement costs.

Step 4: Calculate path costs

For every possible route, the robot adds the movement costs.

$$c(n, n') = \begin{cases} 1, & \text{normal cell} \\ 3, & \text{risky cell} \end{cases}$$

Step 5: Select the lowest-cost route

UCS maintains a priority queue ordered by cumulative path cost $g(n)$. It selects the reachable known node with the lowest cumulative cost.

Step 6: Move and explore

The robot moves toward the selected node, observes its surroundings and updates its map.

Step 7: Replan when necessary

If a newly discovered obstacle blocks the planned route, the robot updates its map and searches for another route.

Step 8: Reach the survivor

The robot continues until it reaches G through a feasible route with minimum total cost based on the discovered map.

Pseudocode:

```
BEGIN

Initialize position S
Initialize known_map
Set total_cost = 0

WHILE survivor G is not reached DO

   Observe adjacent cells

   Update known_map with:
      Free cells
      Obstacles
      Risky zones

   Use UCS to find the lowest-cost
   reachable target on known_map

   IF no safe route is currently known THEN
      Explore an unknown adjacent cell
      IF no safe move is available THEN
         Stop and request assistance
      END IF
   ELSE
      Move to the selected cell

      IF the cell is risky THEN
total_cost = total_cost + 3        ELSE
```

```
            total_cost = total_cost + 1
        END IF
    END IF

    Update the robot's position
    Update known_map
    Replan if new information changes the route

END WHILE

Report survivor location
Report total movement cost

END
```

c) Application of AI Concepts

1. Intelligent Agent

An intelligent agent perceives its environment through sensors and takes actions to achieve a goal. The rescue robot senses nearby rooms, identifies obstacles, chooses movements and attempts to reach the survivor.

2. Rationality

A rational agent selects actions that maximize its expected performance based on the information available. The robot chooses a safe, low-cost route using its current knowledge. When new obstacles are discovered, it changes its plan to improve the chance of reaching the survivor safely.

3. Environment Type

The rescue robot operates in the following environment:

- **Partially observable:** It can observe only adjacent cells.
- **Sequential:** Each movement affects future decisions.
- **Dynamic:** New information may change the robot's map and plan.
- **Stochastic or uncertain:** The actual route conditions and discoveries may be uncertain.
- **Discrete:** The building is represented as a grid of individual cells.
- **Single-agent:** The robot makes its own navigation decisions.

d) Justification for the Chosen Approach

The combination of online search and Uniform Cost Search is suitable for the rescue robot for the following reasons:

1. **Handles incomplete information:** Online exploration allows the robot to discover unknown parts of the building.
2. **Avoids obstacles:** The robot records blocked cells and excludes them from possible routes.
3. **Minimizes total cost:** UCS considers cumulative movement costs, including the additional cost of risky zones.
4. **Supports replanning:** The robot can update its map and choose another route when new obstacles are discovered.
5. **Improves safety:** Risky zones can be assigned higher costs, encouraging the robot to prefer safer alternatives when available.

UCS guarantees the least-cost path on a known graph when all edge costs are nonnegative. In a partially observable environment, however, the robot cannot guarantee the globally optimal route until it has enough information about the relevant paths and costs.

Result: By combining online exploration with UCS, the rescue robot can update its knowledge, avoid obstacles and select a minimum-cost route using the information discovered during navigation.